GUIA VISUAL 094

AUTOMATIZANDO TAREFAS COM CRON E SCRIPTS AGENDADOS

*O backup rodava às duas da manhã havia oito meses. No dia em que o disco morreu, o destino tinha **um** arquivo: o primeiro. O script quebrara na segunda noite — e as 240 execuções seguintes falharam caladas, porque o `>/dev/null 2>&1` que evitava o incômodo diário descartava também o erro.*

094
GUIAS LINUX

SEIS ETAPAS, SEIS MANEIRAS DE FALHAR EM SILÊNCIO

ETAPA	COMO FALHA EM SILÊNCIO	O QUE FECHA A BRECHA
DISPARO	o serviço `cron` não está ativo, ou a linha do `crontab` não tem o `\n` final	`systemctl is-active cron`; editar sempre por `crontab -e`
AMBIENTE	`PATH` mínimo, sem `.bashrc`, `HOME` diferente: `command not found`	caminho absoluto; `PATH`, `LC_ALL` e `cd` declarados no topo do script
CONCORRÊNCIA	a execução anterior ainda roda; duas cópias corrompem o destino	`flock -n 9` num arquivo de trava
EXECUÇÃO	erro no meio e as linhas seguintes rodam mesmo assim — o script ainda diz "concluído"	`set -euo pipefail` e `trap ... EXIT` de limpeza
REGISTRO	`>/dev/null 2>&1` descarta justamente a mensagem de erro	`logger -t` para o journald; nunca descartar o `stderr`
NOTIFICAÇÃO	ninguém é avisado; a falha só aparece no dia em que se precisa do resultado	`MAILTO`, ou um sinal de vida que alerta pela **ausência**

O caso da abertura atravessou a tabela inteira: o erro nasceu na execução, foi descartado no registro e ninguém foi notificado. **Qualquer uma das três defesas, sozinha, teria bastado.** Um comando digitado tem três coisas que a tarefa agendada não tem — você olhando a saída, um ambiente completo e uma execução por vez. Cada ausência dessas é um modo de falha.

O AMBIENTE MÍNIMO DO CRON — E O ESQUELETO QUE RESISTE A ELE

`/usr/local/sbin/backup-diario.sh`

```
#!/usr/bin/env bash
set -euo pipefail                # para no 1º erro
export PATH=/usr/local/bin:/usr/bin:/bin
export LC_ALL=C                  # saída previsível
cd "$(dirname "$0")"
log() { logger -t "$TAG" -p user.info -- "$*"; }
exec 9>"$TRAVA"                  # execução única
flock -n 9 || { log "já rodando"; exit 0; }
tmp=$(mktemp -d)                 # limpeza sempre
trap 'rc=$?; rm -rf "$tmp"; exit $rc' EXIT
[[ -d $ORIGEM ]] || { erro "sem origem"; exit 1; }
rsync -a --link-dest="$DEST/atual" "$ORIGEM/" "$tmp/"
mv -- "$tmp" "$DEST/$(date +%F)"  # quase atômico$ env -i HOME=$HOME
PATH=/usr/bin:/bin ./backup.sh
# imita o cron: se passa aqui, o ambiente não é a causa
```

O QUE O CRON ENTREGA	POR QUE QUEBRA
`PATH=/usr/bin:/bin`	sem `/usr/local/bin` nem `~/.local/bin`: ferramenta de `pipx` ou compilada some
`SHELL=/bin/sh`	é o **dash** no Debian/Kali: `[[ ]]`, arrays e `<( )` dão erro de sintaxe. E nenhum arquivo de inicialização é lido: alias e função do `.bashrc` não existem

Idempotência não é opcional — a tarefa *vai* rodar duas vezes. Teste o estado antes de agir: `grep -qxF "linha" arq || echo "linha" >> arq`. E fuja do `|| true`, que engole "já existia" e "faltou permissão" com a mesma cara.

SER AVISADO — E ESCOLHER ENTRE CRON E TIMER

1 · MAILTO
O `cron` manda a saída por e-mail local — funciona **se** a máquina entrega e-mail, e quase nenhuma entrega. Frágil.

2 · ALERTA NO ERRO
O script avisa quando falha. Melhor — mas com um furo grave: **se nunca chegar a rodar, também nunca alerta**.

3 · SINAL DE VIDA
`curl -fsS "$URL" || true` na **última** linha; o observador externo alerta quando o sinal **não chega**. É o único que pega a tarefa que **deixou de existir**.

SITUAÇÃO	ESCOLHA
tarefa simples e portável; sobreposição resolvida com `flock`	`cron`
máquina desligada no horário, tarefa não pode ser pulada	`anacron` ou timer com `Persistent=true`
depende de outro serviço, precisa de limite de CPU/memória ou de registro estruturado	timer do systemd: herda as diretivas da unit; a saída já vai ao journal

Registrar não é ser avisado. Ninguém lê o journal de uma máquina que funciona. Padronize `logger -t tarefa` desde a primeira linha e consulte com `journalctl -t tarefa --since today`. Se preferir arquivo: `>> /var/log/tarefa.log 2>&1` — **`>>`, não `>`**, e o **`2>&1` depois**, ou o `stderr` some; e arquivo próprio pede `logrotate`, ou você trocou o silêncio pelo disco cheio. Antes de agendar: `bash -n`, `shellcheck`, `env -i` e um `*/5 * * * *` por meia hora.

Nunca escreva `>/dev/null 2>&1` numa tarefa agendada. Para silenciar a saída normal e manter os erros, o correto é `>/dev/null` sozinho: o `stderr` continua chegando e o `cron` avisa. Descartar o `stderr` é decidir de antemão não saber do dia em que a tarefa quebrar — foi a linha que custou oito meses de backup. **E a tarefa roda com o privilégio de quem a agendou:** script no `crontab` do root que seja gravável por usuário comum entrega root a ele na próxima execução. Script de root fica em `/usr/local/sbin`, dono `root`, permissão `755`, nunca onde outro escreve.

RESUMO RÁPIDO

- ✓ **O silêncio é o padrão.** Disparo, ambiente, concorrência, execução, registro e notificação: cada etapa falha sem sintoma até o dia em que você precisa do resultado.
- ✓ **O ambiente do `cron` é mínimo:** `PATH=/usr/bin:/bin`, `SHELL=/bin/sh` (o *dash*) e nenhum arquivo de inicialização. Descubra o real com `* * * * * /usr/bin/env > /tmp/amb.txt`.
- ✓ `set -euo pipefail` + `flock` + `trap ... EXIT` é o esqueleto mínimo: parar no 1º erro, execução única, limpeza **inclusive** no caminho de falha.
- ✓ **Idempotência não é opcional** — teste o estado antes de agir, e nunca mascare erro com `|| true`.
- ✓ **O alerta que pega o pior caso funciona pela ausência:** só o sinal de vida detecta a tarefa que *deixou de existir* — `crontab` apagado, serviço desativado, máquina desligada.
- ✓ **Teste com `env -i` antes de agendar.** Um script que você só viu funcionar no seu terminal não foi testado para o `cron`.

NO KALI

O Kali costuma ser uma VM que fica desligada — e isso quebra o `cron` por inteiro: ele só dispara se a máquina estiver ligada no minuto exato, então a tarefa das duas da manhã *nunca* roda numa VM que você liga às nove, e nenhum erro avisa. Para manutenção e atualização de bases de ferramentas, o que você quer é o `anacron` — que executa o que foi perdido assim que a máquina liga — ou um timer com `Persistent=true`. **Automatizar `apt update` é seguro; `apt full-upgrade` sem supervisão, não:** num repositório *rolling*, uma atualização pode trocar kernel, driver gráfico ou biblioteca no meio da noite e deixar a VM sem interface pela manhã. Índices sozinhos, upgrades com você olhando. **Como praticamente nenhum Kali tem MTA, o `MAILTO` não entrega nada** e a saída se perde: `logger -t` desde a primeira linha resolve sem instalar nada, e `journalctl -t tarefa --since today` vira a consulta única. **Coleta periódica em teste autorizado é rotina — e o escopo tem de estar dentro do script**, porque a tarefa *continua rodando depois que o engajamento termina*: um scan esquecido no `crontab`, batendo num alvo após o fim do contrato, deixa de ser autorizado, e uso não autorizado é crime (Lei 12.737/2012). Grave a data de término no script e remova a entrada no encerramento. **Auditar o `crontab` do root e o `/etc/cron.d/` é parada obrigatória** em revisão de sistema comprometido: é um dos lugares mais comuns de persistência de invasor. **Fora do Debian:** o Fedora/RHEL usa o `cronie`, mesma sintaxe, `anacron` embutido; lá o `/bin/sh` é o Bash em modo POSIX, então *bashismo* passa e falha de volta no Debian (teste no `dash`), e o SELinux bloqueia sem sintoma: `ausearch -m avc -ts recent` quando a tarefa "não faz nada".